



Fluxion

Build the machinery behind a neural network, then measure it.

Fluxion is a readable deep learning engine built from first principles. It owns its tensors, dynamic computation graph, backward rules, trainable layers and optimizers, then composes those pieces into a small GPT-style model. PyTorch is an independent numerical reference rather than the training engine underneath Fluxion.

The problem

Understanding a model's training behavior requires seeing how gradients accumulate, how shapes broadcast and where complete training steps spend time. Moving an operator into native code is only useful if the actual workload improves.

What you can do

Tensors and reverse-mode autograd Array operations build a dynamic graph. Backward traversal accumulates all contributions to a shared input and reduces broadcast gradients to their original shapes.	Trainable modules Linear, activation, normalization and embedding layers compose with losses, recursive parameter discovery, SGD and Adam.
A small causal Transformer Multi-head causal attention, learned positions and pre-norm residual blocks produce GPT logits for next-token training and generation.	Explicit backend experiments NumPy is the default path. An opt-in C++/BLAS Linear operator and experimental standalone CUDA Linear kernels expose native boundaries.

How it works

01 / Forward Tensor operations record parent inputs and local backward rules while producing a model output and loss.	02 / Differentiate A topological reverse traversal computes local derivatives and accumulates each parent contribution.
03 / Update SGD or Adam changes the trainable arrays; the same loop supports the toy network and GPT example.	04 / Check and measure Independent PyTorch implementations check outputs and gradients; full training-step benchmarks retain raw samples.

TECHNOLOGY / Python · NumPy · C++ · BLAS · CUDA · PyTorch

Decisions that shape the system

Readable correctness first

The graph and derivatives remain inspectable. For $y = x*x + x$ at $x = 3$, the shared x receives $3 + 3 + 1$, producing gradient 7.

Reference both directions

Validation copies the same inputs and parameters into independent PyTorch models and checks both forward values and backward gradients.

Measure the whole step

Benchmarks include forward, loss, backward and update. Operator-level work does not automatically translate into a faster model.

Observed validation

The October 4 Linux CPU/native checkpoint passed 72 tests with 2 CUDA skips. All five PyTorch reference reports passed. The retained workload measurements include 2,000 timed samples. The seeded tiny network's loss fell from 0.256 to $7.81e-13$ after 1,000 updates.

Scope of this evidence

Native Linear was slower on the measured Linux host. CUDA was not exercised in that checkpoint; device-resident GPT training is future work. The tiny repeated-text GPT example is a learning exercise, not evidence of broad language understanding.

Explore the implementation

Source code and runnable examples

github.com/michaelbawuah/Fluxion

Autograd implementation

github.com/michaelbawuah/Fluxion/blob/main/src/fluxion/autograd.py

Small GPT model

github.com/michaelbawuah/Fluxion/blob/main/src/fluxion/transformer/gpt.py

Independent PyTorch validation

github.com/michaelbawuah/Fluxion/blob/main/validation/validate_pytorch.py

Benchmark methodology and results

github.com/michaelbawuah/Fluxion/blob/main/docs/benchmarks/results.md

This brief explains the project using its public source and retained validation records. It is an engineering overview, not a research publication.